

Arquitectura de Simple Stock Flow: artefactos, decisiones y atributos de calidad

Jesus Ariel Gonzalez Bonilla

Simple Stock Flow — sistema de inventario y ventas

21 de septiembre de 2026

Nota de autor

Este documento describe la arquitectura de un sistema de inventario y ventas de extremo a extremo, desarrollado íntegramente por el autor. Cada afirmación se comprobó contra la fuente que la sostiene —el código fuente, los archivos de proyecto, la configuración de despliegue y el motor de base de datos en ejecución—, y las que dependen de un estado que puede cambiar llevan escrita su fecha de comprobación.

No hay financiación que declarar ni conflicto de intereses que informar. La correspondencia relativa a este documento se dirige al autor.

Resumen

Este documento describe la arquitectura de Simple Stock Flow, un sistema de extremo a extremo para la gestión de un catálogo de productos y el registro de ventas. El sistema se reparte en tres unidades desplegables: una que orquesta los contenedores, un servicio en .NET 8 organizado como hexágono de puertos y adaptadores, y un portal en Angular 20 que repite esa misma forma. El documento recorre los artefactos de cada unidad, documenta la comunicación entre ellas — qué protocolo cruza cada frontera, quién autoriza cada tramo y en qué punto se traduce un error de dominio a un código de respuesta—, registra las cuatro decisiones estructurales con las alternativas que se estudiaron y el motivo por el que se rechazaron, y declara los atributos de calidad que el sistema persigue junto con el precio que paga por cada uno. Una tabla de trazabilidad ata cada regla de negocio y cada requisito no funcional con consecuencia estructural al artefacto que lo sostiene, y un registro de riesgos recoge, con dueño y vía de salida, lo que se decidió no hacer. Ocho figuras sostienen la explicación: la topología desplegada, el sistema en ejecución, la estructura interna de los repositorios, el hexágono duplicado, el grafo real de dependencias entre proyectos, la secuencia de la operación donde se concentra el riesgo, el modelo de datos con las garantías que impone el motor y el camino completo de una petición.

Palabras clave: arquitectura hexagonal, puertos y adaptadores, regla de dependencia, atributos de calidad, decisiones de arquitectura, trazabilidad, .NET 8, Angular, contenedores

Índice

Resumen.....	2
Alcance, público y método de verificación.....	6
Convenciones de este documento.....	7
Panorama del sistema.....	8
Las tres unidades desplegables.....	10
La estructura interna de las unidades.....	14
Artefactos del servicio.....	16
La solución y su grafo de referencias.....	16
El dominio.....	20
La capa de aplicación.....	20
Puertos entrantes.....	21
Puertos salientes.....	21
Servicios de aplicación.....	22
Tipos comunes de la capa.....	25
El adaptador entrante.....	25
Los adaptadores salientes.....	26
Persistencia.....	26
Seguridad.....	28
Almacenamiento de binarios.....	29
La raíz de composición y el arranque.....	29
Los artefactos de prueba.....	30
El artefacto de imagen.....	31
Artefactos del portal.....	31
El dominio del portal.....	31
La capa de aplicación del portal.....	31
La capa de infraestructura del portal.....	32
La capa de presentación.....	32
Artefactos de despliegue del portal.....	32
Artefactos de infraestructura.....	33
La composición de contenedores.....	33
La configuración por entorno.....	34
El guion de verificación.....	34
Los artefactos de demostración.....	35
Volúmenes y red.....	36
La comunicación entre las unidades.....	36
Fronteras y protocolos.....	39
Autenticación y autorización.....	39
La ruta pública de binarios.....	40

La traducción de errores.....	40
Decisiones de arquitectura.....	41
Decisión 1 · La propiedad del esquema.....	42
Decisión 2 · El control de concurrencia sobre las existencias.....	44
Decisión 3 · La retirada de un producto del catálogo.....	45
Decisión 4 · El reporte agregado sobre atributos congelados.....	47
Atributos de calidad: qué se persigue y qué se paga.....	49
Fiabilidad e integridad.....	49
Fidelidad histórica.....	50
Seguridad y confidencialidad.....	50
Mantenibilidad.....	51
Rendimiento y escala.....	52
Requisitos no funcionales y dónde se satisfacen.....	53
Trazabilidad: de la regla al artefacto.....	54
Riesgos abiertos y límites aceptados.....	56
Deuda aceptada y decisiones aplazadas.....	57
Consideraciones finales.....	58
Referencias.....	60

Índice de tablas

Tabla 1. Reparto de responsabilidades entre las tres unidades desplegables.....	10
Tabla 2. Puertos entrantes de la capa de aplicación.....	21
Tabla 3. Puertos salientes de la capa de aplicación y adaptador que los implementa.....	22
Tabla 4. Migraciones del esquema, en orden de aplicación.....	28
Tabla 5. Claves de configuración declaradas en el archivo de ejemplo.....	34
Tabla 6. Qué cruza cada frontera del sistema.....	39
Tabla 7. Correspondencia entre causa y código de respuesta.....	41
Tabla 8. Las cuatro decisiones de arquitectura, con la alternativa principal que cada una descartó.....	42
Tabla 9. Realización de los requisitos no funcionales.....	54
Tabla 10. Trazabilidad de las reglas y los requisitos con consecuencia estructural.....	55
Tabla 11. Registro de riesgos de arquitectura.....	57

Índice de figuras

Figura 1. Topología desplegada del sistema Simple Stock Flow.....	9
Figura 2. El sistema desplegado: el catálogo servido por el portal a través del único puerto publicado.....	12
Figura 3. Estructura en disco: las tres unidades desplegables, la documentación y las herramientas de demostración.....	15
Figura 4. El hexágono, repetido en las dos unidades de código.....	17
Figura 5. Grafo de referencias entre los diez proyectos de la solución.....	19
Figura 6. Secuencia del registro de una venta: invariante, congelación, confirmación atómica y reintento acotado.....	24
Figura 7. El modelo de datos: cinco tablas, cuatro claves foráneas y lo que el motor garantiza por su cuenta.....	27
Figura 8. Camino completo de una petición, de la pantalla al motor.....	38

Arquitectura de Simple Stock Flow: artefactos, decisiones y atributos de calidad

Simple Stock Flow resuelve un problema acotado —mantener un catálogo de productos, registrar ventas con sus líneas, emitir un reporte por rango de fechas y autenticar a quien opera— con una disposición que no es la mínima posible para ese problema, sino la que permite explicarlo, revisarlo y cambiarlo sin adivinar. Esa diferencia es el objeto de este documento: no qué hace el sistema, sino de qué piezas está hecho, por qué cada pieza está donde está, qué se decidió no hacer y qué cuesta cada garantía que ofrece.

La disposición elegida es la arquitectura hexagonal, también llamada de puertos y adaptadores (Cockburn, 2005). Su propiedad central es que la lógica de negocio no depende de ninguna tecnología: quien invoca al núcleo lo hace a través de un puerto entrante, y aquello de lo que el núcleo depende —una base de datos, un reloj, un almacén de binarios— entra por un puerto saliente que el núcleo declara y que otro implementa. La consecuencia práctica, formulada por Martin (2017) como regla de dependencia, es que las referencias apuntan siempre hacia dentro y nunca hacia fuera.

Lo que distingue a este sistema de un diagrama con esa forma es que la regla está impuesta por el compilador y no por la disciplina de quien escribe. Cada capa es un proyecto independiente con sus propias referencias declaradas, y una referencia que violara la regla no compilaría. El mismo criterio gobierna el resto del documento: allí donde una propiedad arquitectónica se enuncia, se nombra el mecanismo que la hace fallar de forma ruidosa cuando se incumple, y allí donde no existe tal mecanismo se dice que no existe.

Alcance, público y método de verificación

El público de este documento es técnico: quien tenga que mantener el sistema, evaluarlo o extenderlo. Se supone familiaridad con el protocolo web, con contenedores y con al menos un lenguaje orientado a objetos; no se suponen conocimientos previos del sistema.

El alcance es la forma del sistema, que cambia despacio: las unidades desplegables, los artefactos que las componen, la comunicación entre ellas, las decisiones que las explican y los atributos de calidad que persiguen. Queda fuera, deliberadamente, el avance de la construcción —qué está terminado hoy y qué no—, que se desactualiza en cuestión de horas y convertiría a este documento en una segunda fuente de verdad. Una segunda fuente miente antes que la primera.

Método. Cada afirmación se comprobó leyendo la fuente que la sostiene: los archivos de proyecto para el grafo de dependencias, el archivo de composición de contenedores para la topología y el orden de arranque, la configuración del servidor intermediario para el enrutado, el código de arranque del servicio para la secuencia de migración y siembra, y **el propio motor de base de datos en ejecución** para las restricciones, las claves foráneas y los índices que se describen. Las afirmaciones que dependen de un estado que puede cambiar llevan su fecha de comprobación.

Convenciones de este documento

Los identificadores de código, las rutas de archivo, los nombres de contenedor y los fragmentos de configuración aparecen en fuente monoespaciada, conforme a la recomendación del Manual de Publicaciones para el código de computadora (American Psychological Association [APA], 2020).

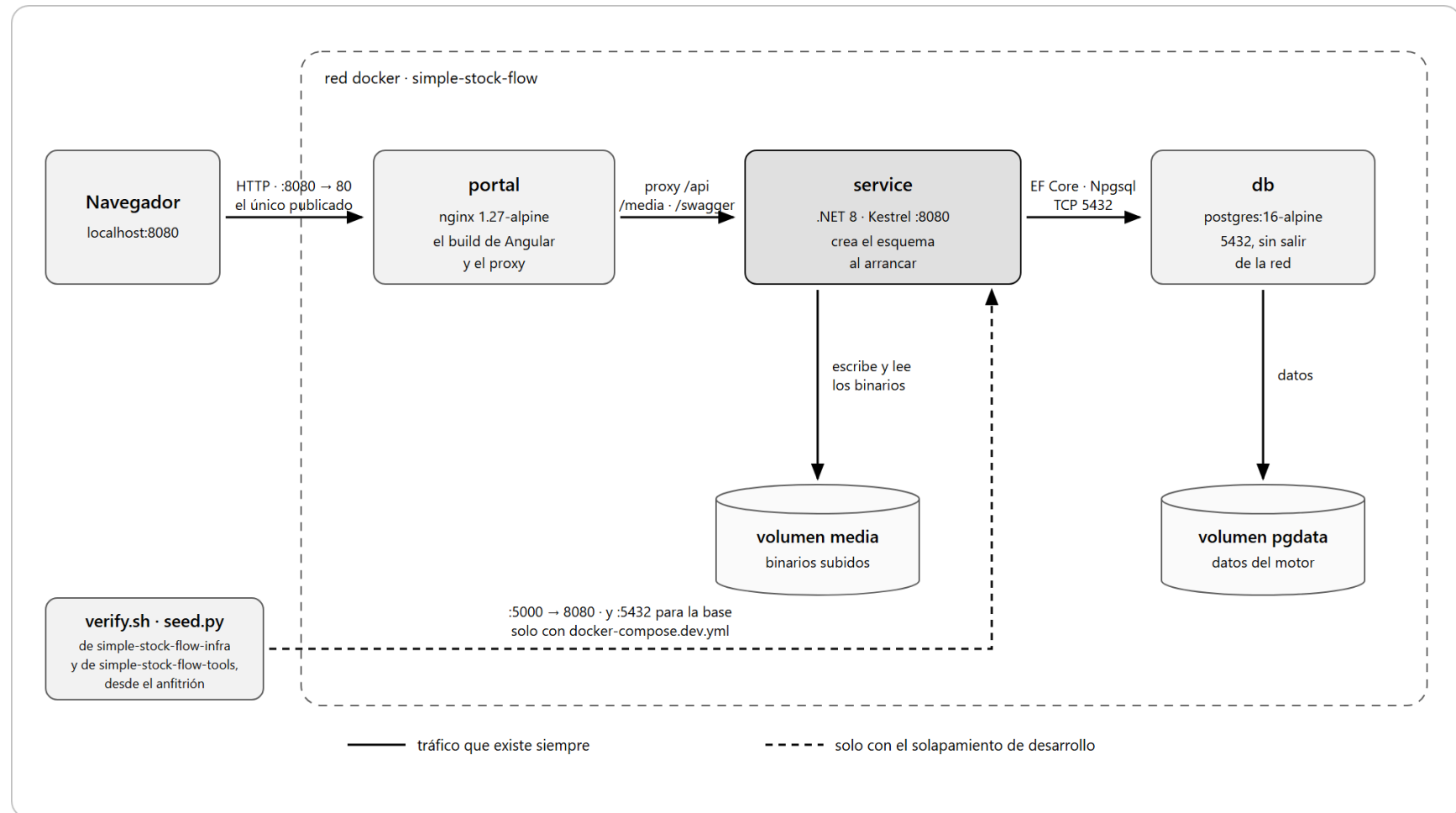
Las reglas de negocio del sistema se numeran de RN-01 a RN-12, sus requisitos no funcionales de RNF-01 a RNF-08 y los criterios que los verifican con la forma CA-nn.n. Esos

códigos se usan aquí para atar una afirmación a la exigencia que la obliga, y no como referencia a un documento externo: cada uno se enuncia allí donde se cita.

El sistema mantiene una frontera idiomática estricta que conviene conocer al leer los nombres que aquí se citan: lo que ve quien **usa** el sistema —interfaz, mensajes de error dirigidos a una persona, rutas de la dirección web— está en español, y lo que ve quien lo **mantiene** —identificadores, comentarios, nombres de prueba, registros de ejecución— está en inglés. Por eso conviven en el mismo párrafo `ProductCatalogService` y la ruta `/productos`.

Panorama del sistema

El sistema se despliega como tres contenedores en una red privada y publica un solo puerto al exterior (véase la Figura 1). Esa asimetría no es un detalle de operación: es la primera decisión de arquitectura que conviene tener en la cabeza, porque explica por qué el portal hace de intermediario y por qué el servicio no necesita estar expuesto para que el sistema funcione.

Figura 1*Topología desplegada del sistema Simple Stock Flow*

Nota. Las líneas continuas representan tráfico que existe en cualquier despliegue; las discontinuas, tráfico que solo existe cuando se aplica el solapamiento de desarrollo `docker-compose.dev.yml`. Elaboración propia a partir de `docker-compose.yml`, `docker-compose.dev.yml` y `nginx.conf`.

Las tres unidades desplegables

`simple-stock-flow-infra` levanta el conjunto: define la red, los volúmenes, las comprobaciones de salud y las variables de entorno. `simple-stock-flow-api` aporta el dominio, la interfaz de programación y —esto es lo que lo distingue— el esquema completo de la base de datos en forma de migraciones. `simple-stock-flow-portal` aporta la interfaz de usuario y sus propios casos de uso, y no almacena nada: su única dependencia externa es la interfaz de programación.

Los tres se clonan como hermanos dentro de un mismo directorio, porque el archivo de composición construye las imágenes desde rutas relativas (`../simple-stock-flow-api` y `../simple-stock-flow-portal`). **La disposición en disco forma parte del contrato de despliegue** y no es una preferencia de organización: mover uno de los dos o anidarlos dentro de otro deja la construcción sin contexto y falla con un mensaje que no explica la causa.

La Tabla 1 reparte las responsabilidades, y conviene leer con atención la tercera columna: lo que cada unidad **no** hace es lo que impide que las tres se conviertan en una.

Tabla 1

Reparto de responsabilidades entre las tres unidades desplegables

Unidad	Es responsable de	No es responsable de
<code>simple-stock-flow-infra</code>	Contenedores, red, volúmenes, comprobaciones de salud y orquestación del conjunto	El esquema de datos. No contiene una sola sentencia de definición de datos
<code>simple-stock-flow-api</code>	Reglas de negocio, interfaz de programación y el esquema completo, aplicado por migraciones	La orquestación del despliegue
<code>simple-stock-flow-portal</code>	Interfaz de usuario y los casos de uso propios de esa interfaz	El almacenamiento. Solo conversa con la interfaz de programación

Nota. La propiedad del esquema por parte del servicio es la decisión estructural que sostiene el reparto, y se desarrolla más adelante como Decisión 1.

Esa topología no es una intención: se despliega y se abre. La Figura 2 es el catálogo servido por el portal a través del único puerto publicado, con las filas que la interfaz de programación devuelve y las imágenes que salen del volumen de binarios.

Figura 2

El sistema desplegado: el catálogo servido por el portal a través del único puerto publicado

SSF Simple Stock Flow

Productos

Nueva venta

Ventas

Reportes

Nuevo vendedor

Carrito 0

admin
Administrador

Salir

Productos

23 productos en el catálogo

Nuevo producto

NOMBRE	CATEGORÍA	PRECIO	STOCK	ACCIONES
 Alicata universal 8"	Herramientas	\$ 32.400	20	Editar
 Bombilla LED 9 W luz cálida	Electricidad	\$ 9.800	80	Editar
 Brocha 3" de cerda natural	Pinturas	\$ 12.800	32	Editar
 Cable encauchetado 3x14, por metro	Electricidad	\$ 6.400	235	Editar
 Candado de seguridad 50 mm	General	\$ 28.700	24	Editar
 Cinta de teflón 12 m	Fontanería	\$ 2.900	144	Editar
 Escalera tijera 5 pasos en aluminio	General	\$ 219.000	0	Editar
 Esmalte sintético negro 1/4	Pinturas	\$ 34.600	11	Editar
 Extensión múltiple 6 tomas	Electricidad	\$ 46.900	11	Editar
 Flexómetro 5 m	Herramientas	\$ 18.500	36	Editar

Nota. Captura del sistema en ejecución el 21 de septiembre de 2026, convertida a escala de grises como el resto de las figuras de este documento. La barra superior, la paginación del catálogo y el estado de existencias corresponden a la interfaz descrita en el manual de usuario.

La estructura interna de las unidades

Vistas desde fuera, las tres unidades reparten responsabilidades. Vistas por dentro, lo primero que salta es que **las dos unidades de código tienen la misma silueta** (véase la Figura 3): un directorio de dominio, uno de aplicación y uno de adaptadores o de infraestructura, con la capa de presentación aparte en el caso del portal. Esa coincidencia no es casual y se explica pieza por pieza en los dos capítulos siguientes.

La figura recoge además dos directorios que **no participan en ninguna construcción**: el de la documentación y el de las herramientas de demostración. Ningún artefacto de código depende de ellos, y por eso pueden moverse sin romper nada. Que ambos vivan fuera de las unidades de código es el reverso del motivo por el que estas son hermanas: un documento transversal —o una herramienta opcional— alojado dentro de una de ellas apunta a rutas que dejan de existir en cuanto esa unidad se clona sola. En disco hay un tercer directorio de esa misma clase, `simple-stock-flow-page`, que publica los tres entregables de este proyecto como página estática y tampoco participa en ninguna construcción; la figura lo deja fuera a propósito, porque su alcance es el que anuncia su título: las tres unidades desplegadas, la documentación y las herramientas de demostración.

Figura 3

Estructura en disco: las tres unidades desplegables, la documentación y las herramientas de demostración

simple-stock-flow-docs/ └─ entregables/ └─ arquitectura-simple-stock-flow-apa7.* └─ manual-tecnico.* └─ manual-usuario.* └─ plantillas/ └─ arquitectura-apa7/ └─ manual-tecnico/ └─ manual-usuario/ └─ lib/ └─ diagramas/ └─ capturas/ · README.md	la documentación 3 documentos, en .docx y .pdf fuente de los tres formato común 9 SVG
simple-stock-flow-infra/ └─ docker-compose.yml └─ docker-compose.dev.yml └─ .env.example └─ verify.sh └─ README.md	orquesta el conjunto .env ignorado
simple-stock-flow-tools/ └─ catalogue.py └─ seed.py └─ make_pictures.py └─ img/ └─ README.md	demostración · opcional siembra por HTTP 22 imágenes
simple-stock-flow-api/ └─ src/ └─ domain/ └─ Catalog/ · Sales/ └─ Identity/ · Common/ └─ ValueObjects/ └─ application/ └─ Ports/Inbound/ └─ Ports/Outbound/ └─ Services/ └─ Common/ └─ adapters/ └─ inbound/api/ └─ outbound/ └─ persistence/ └─ security/ └─ storage/ └─ bootstrap/ └─ Composition/ └─ Program.cs └─ tests/ └─ Domain.UnitTests/ └─ Application.UnitTests/ └─ Adapters.IntegrationTests/ └─ Directory.Build.props └─ Dockerfile └─ SimpleStockFlow.sln └─ README.md	.NET 8 · 10 proyectos 0 referencias 6 10 5 5 REST 9 migraciones JWT · PBKDF2 binarios PortBindings.cs sin dobles con dobles motor real una advertencia rompe 10 proyectos
simple-stock-flow-portal/ └─ src/app/ └─ domain/ └─ models/ · policies/ └─ value-objects/ └─ application/ └─ ports/ └─ use-cases/ └─ state/ └─ infrastructure/ └─ http/ · dto/ └─ mappers/ └─ interceptors/ └─ providers.ts └─ presentation/ └─ features/ └─ guards/ └─ layout/ · shared/ └─ app.routes.ts └─ src/environments/ └─ nginx.conf └─ proxy.conf.json └─ Dockerfile └─ angular.json └─ README.md	Angular 20 sin Angular 4 9 3 almacenes api.dto.ts token · errores composición 5 dominios sesión · admin rutas en español 5 localizaciones ng serve node → nginx

Nota. Se han omitido los directorios generados —dependencias descargadas y salidas de compilación—, los archivos de licencia y de configuración auxiliar, y, en el repositorio de la documentación, tres directorios que esta exposición no recorre: la especificación, los traspasos y el enunciado. Los recuentos anotados al margen son los reales, no los previstos. Ni la documentación ni las herramientas de demostración participan en ninguna construcción: ningún artefacto de código depende de ellas. Elaboración propia a partir de la inspección del árbol de directorios el 21 de septiembre de 2026.

En el servicio, cada carpeta de primer nivel dentro de `src/` es un proyecto independiente con sus propias referencias declaradas; en el portal, cada carpeta de primer nivel dentro de `src/app/` es una capa cuyo contenido no puede importar hacia fuera. Los recuentos anotados — seis puertos entrantes, diez salientes, cuatro puertos y nueve casos de uso en el portal— son los que las tablas de los apartados siguientes detallan uno a uno.

Artefactos del servicio

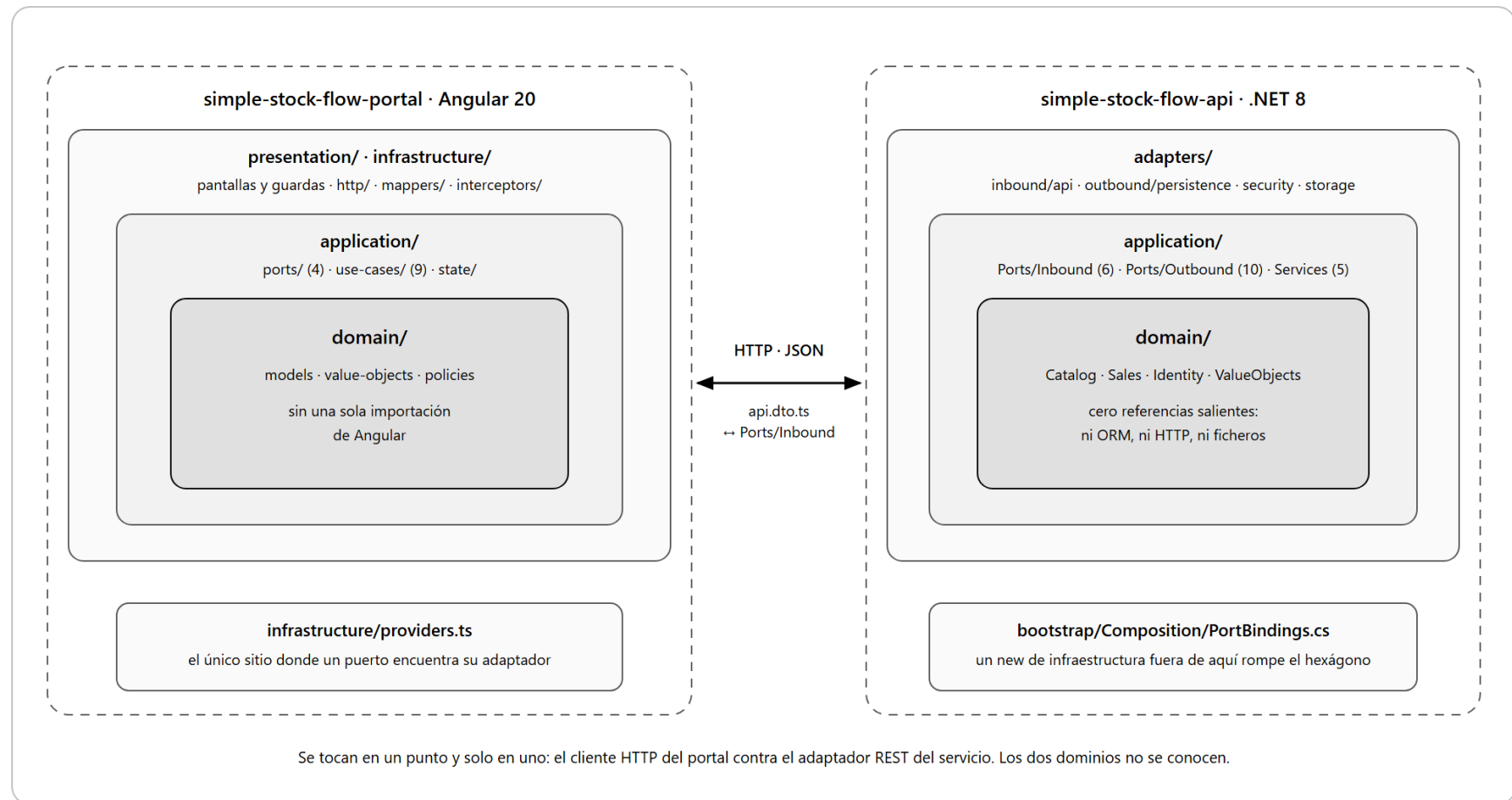
El servicio es una solución de .NET 8 compuesta por diez proyectos: siete de código y tres de prueba. Que sean proyectos separados, y no carpetas dentro de uno solo, es el mecanismo que convierte la regla de dependencia en una restricción del compilador.

La solución y su grafo de referencias

El portal repite la disposición del servicio (véase la Figura 4), y las dos se tocan en un único punto.

Figura 4

El hexágono, repetido en las dos unidades de código

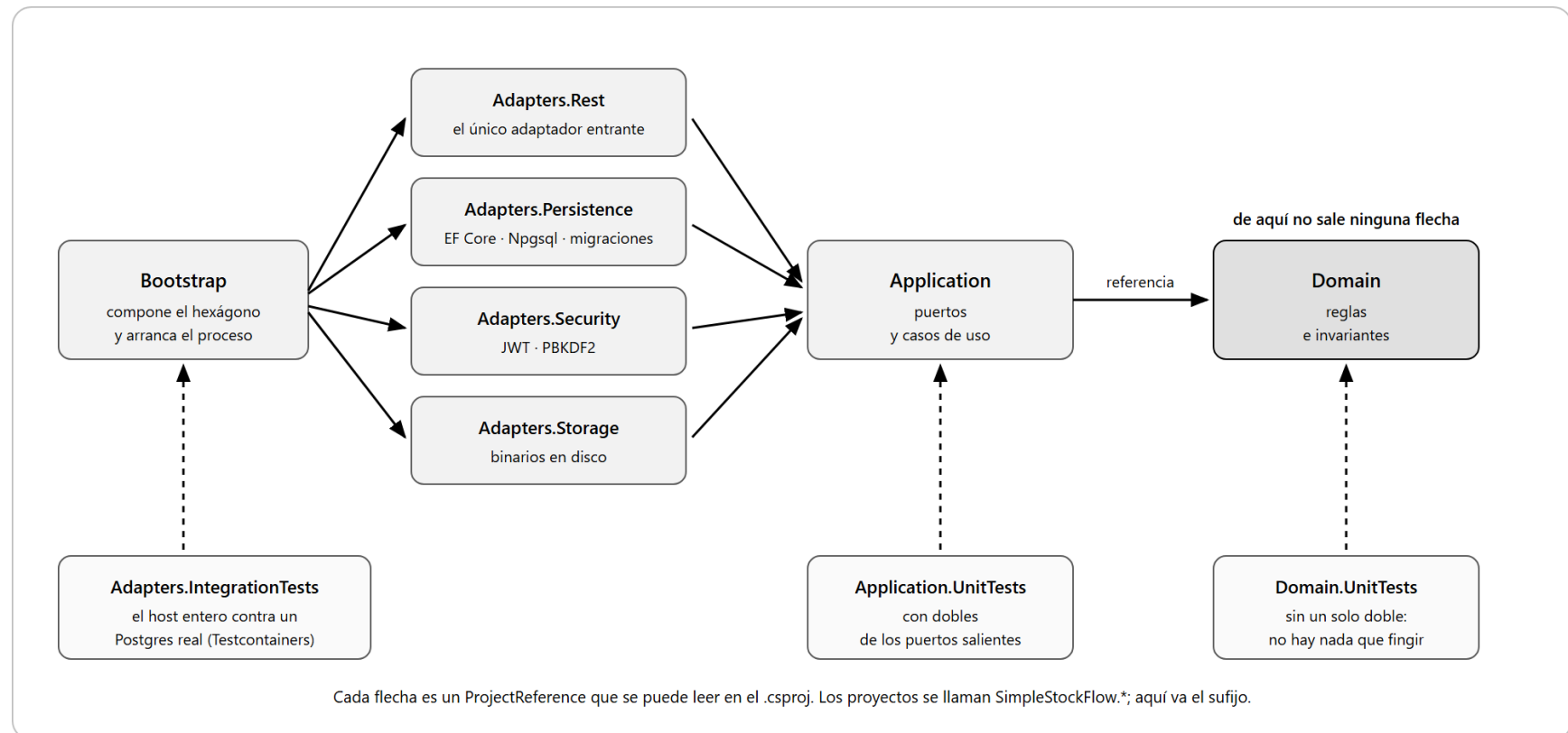


Nota. Los dos se tocan en un único punto: el cliente de transporte del portal contra el adaptador entrante del servicio. Elaboración propia a partir de la estructura de proyectos de ambas unidades.

Dentro del servicio, la regla de dependencia no es una intención sino un grafo que se puede leer sin ejecutar nada (véase la Figura 5). Cada flecha es una referencia declarada en un archivo de proyecto; del dominio no sale ninguna, y ningún adaptador referencia a otro adaptador. Una referencia que violara la regla no compilaría, de modo que la única forma de romper la arquitectura es un cambio que nadie podría confundir con un descuido.

Figura 5

Grafo de referencias entre los diez proyectos de la solución



Nota. Cada flecha es una referencia de proyecto declarada en un archivo .csproj y verificable sin ejecutar el sistema. De Domain no sale ninguna. Los proyectos se llaman SimpleStockFlow.*; en la figura consta el sufijo. Elaboración propia.

El dominio

El proyecto de dominio agrupa diez archivos y no referencia nada, ni siquiera un paquete de terceros. Contiene las raíces de agregado `Product`, `Sale` y `User`, la entidad `SaleItem` subordinada a la venta, la entidad de referencia `Category`, y los objetos de valor `Money` y `Quantity`. Los tipos base —`Entity`, `AggregateRoot` y `DomainException`— viven en `Common`.

La distinción entre raíz de agregado y entidad subordinada sigue el criterio de Evans (2003): una línea de venta no existe fuera de su venta, no se modifica por separado y no se referencia desde fuera. Esa decisión tiene una consecuencia visible en la interfaz de programación: no hay operación alguna que edite una línea de venta, y la venta no admite modificación una vez registrada. Es la regla `RN-07`, y no la sostiene una comprobación sino **una ausencia**: no existe operación que lo permita, ni en el agregado, ni en el puerto de ventas, ni en las rutas publicadas.

Los objetos de valor no son adorno. `Money` fija la moneda como constante del dominio en lugar de como columna, porque el sistema es monomonedado por decisión de alcance y `RN-09` exige que todos los importes de una venta compartan divisa; el objeto **lo hace oír**: si un precio llega en otra divisa, la creación y el cambio de precio del producto lanzan, en lugar de quedarse con el importe y descartar la moneda en silencio, que es como una cifra en una divisa se convierte en otra sin que nadie lo note. `Quantity` impide construir una cantidad inválida. Y la invariante que más consecuencias tiene —`RN-01`, que las existencias nunca queden negativas— se comprueba en el dominio al retirar, y además está respaldada por una comprobación del motor, de modo que ninguna ruta de escritura puede eludirla.

La capa de aplicación

La capa de aplicación declara los puertos y orquesta los casos de uso. No sabe nada del protocolo web, del mapeo objeto-relacional ni del sistema de archivos: ninguna de sus firmas menciona un tipo de esas tecnologías, y esa propiedad se comprueba con una búsqueda de texto.

Puertos entrantes

Son seis interfaces (véase la Tabla 2), y representan todo lo que el sistema sabe hacer. Cualquier adaptador entrante —hoy solo el de transporte web, mañana una línea de órdenes o un proceso programado— entra por aquí y no por otro sitio.

Tabla 2

Puertos entrantes de la capa de aplicación

Puerto	Caso de uso que expone
<code>IAuthenticate</code>	Autenticar a un usuario y registrar uno nuevo con un rol determinado
<code>IManageProducts</code>	Crear, actualizar, eliminar, obtener y listar productos de forma paginada; adjuntar la imagen de un producto; y listar las categorías de referencia
<code>ISaleSale</code>	Registrar una venta con sus líneas
<code>ISaleSale</code>	Consultar una venta concreta y listar las ventas
<code>ISaleSalesReport</code>	Emitir el reporte agregado para un rango de fechas
<code>ISaleProvisionAdministrator</code>	Crear el administrador inicial. Es un acto de despliegue y no una operación del sistema en marcha, y por eso vive en un puerto aparte que el adaptador web no tiene inyectado

Nota. Las categorías se listan desde el puerto de catálogo y no desde un puerto propio, porque son datos de referencia de solo lectura: ningún caso de uso las crea, renombra ni elimina.

Puertos salientes

Son diez interfaces (véase la Tabla 3), y representan todo aquello de lo que el núcleo depende. Cada una la implementa un adaptador distinto, y ninguna menciona la tecnología que la satisface.

Tabla 3

Puertos salientes de la capa de aplicación y adaptador que los implementa

Puerto	Responsabilidad	Implementación
<code>IProductRepository</code>	Lectura y escritura de productos	<code>EfProductRepository</code>
<code>ISaleRepository</code>	Lectura y escritura de ventas	<code>EfSaleRepository</code>
<code>ICategoryRepository</code>	Lectura de categorías de referencia	<code>EfCategoryRepository</code>
<code>IUserRepository</code>	Lectura y escritura de usuarios	<code>EfUserRepository</code>
<code>ISalesReportQuery</code>	Lectura agregada del reporte, resuelta en el motor	<code>SqlSalesReportQuery</code>
<code>IUnitOfWork</code>	Confirmación atómica de los cambios	<code>EfUnitOfWork</code>
<code>IFileStorage</code>	Guardado de binarios bajo una clave opaca	<code>LocalFileStorage</code>
<code>IPasswordHasher</code>	Derivación y verificación de contraseñas	<code>Pbkdf2PasswordHasher</code>
<code>ITokenGenerator</code>	Emisión del testigo de sesión	<code>JwtTokenGenerator</code>
<code>IClock</code>	Instante actual, inyectable	<code>SystemClock</code>

Nota. Los puertos de repositorio y de unidad de trabajo siguen los patrones homónimos descritos por Fowler (2002). `IClock` existe para que el instante de una venta sea sustituible en las pruebas sin congelar el reloj del sistema.

Servicios de aplicación

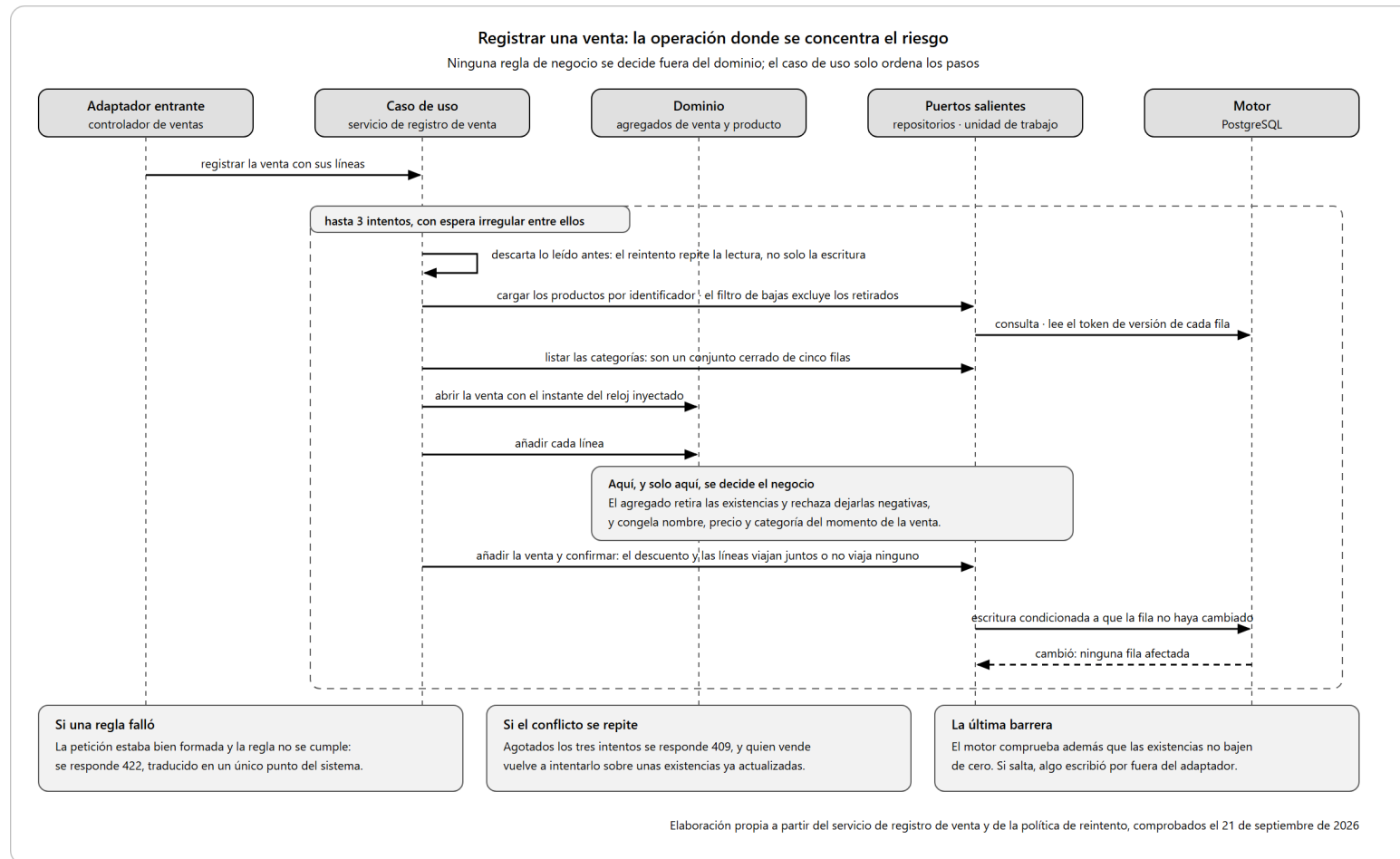
Cinco clases implementan los seis puertos entrantes: `AuthenticationService` —la única que cubre dos, `IAuthenticate` e `IProvisionAdministrator`—, `ProductCatalogService`, `PlaceSaleService`, `GetSaleService` y `SalesReportService`. Ninguna decide una regla de negocio: la delegan toda en el agregado, y una condición de negocio que apareciera en una de ellas pertenecería al dominio.

El más delicado es `PlaceSaleService`, donde se concentran tres propiedades que las pruebas ejercen por separado (véase la Figura 6): atomicidad —el descuento de existencias y las líneas se confirman juntos o ninguno, criterio CA-04.6—, aislamiento —dos ventas simultáneas del último ejemplar no dejan inventario negativo— y congelación —el nombre, el precio y la categoría de la línea no siguen al catálogo, criterio CA-06.4—. El reintento repite el caso de uso

completo, lectura incluida, y no solo la confirmación: reintentar únicamente la escritura reaplicaría un descuento calculado sobre unas existencias viejas, que es el error más probable al implementar este patrón.

Figura 6

Secuencia del registro de una venta: invariante, congelación, confirmación atómica y reintento acotado



Nota. La operación donde se concentra el riesgo del sistema. Elaboración propia a partir del servicio de registro de venta, de la política de reintento y del agregado de venta, comprobados el 21 de septiembre de 2026.

El servicio de reporte merece mención aparte porque **no carga ventas**: recibe filas ya agregadas por el motor a través de `ISalesReportQuery`. Traer el rango completo a memoria para sumarlo allí está declarado defecto y no alternativa, y el diseño del puerto es lo que impide escribirlo de la manera equivocada.

Tipos comunes de la capa

`Common` reúne cinco piezas transversales: `DateRange`, que normaliza los extremos de un rango; `PageRequest` y `PagedResult<T>`, que acotan el tamaño de página —veinte por omisión, cien como tope— y describen el resultado paginado; `ConcurrencyConflictException`, que señala el choque de versiones; y `ConflictRetryPolicy`, que encapsula el reintento de tres intentos con espera irregular y se registra como componente único porque no guarda conversación, solo una función de espera.

El tope de tamaño de página se aplica **recortando y no rechazando**: una petición de mil elementos se sirve con cien, y la respuesta devuelve el tamaño servido y no el pedido. Devolver el pedido descuadraría el número de páginas que el cliente calcula, y su paginador saltaría filas.

El adaptador entrante

El adaptador de transporte web reúne cinco controladores —autenticación, productos, categorías, ventas y reportes—, los contratos de petición y respuesta propios del transporte, un enlazador de los extremos del rango de fechas y, sobre todo, un filtro de traducción de excepciones.

Ese filtro es el único lugar del sistema donde una excepción de dominio se convierte en un código de respuesta. La regla operativa que impone es sencilla de verificar: si aparece un bloque de captura de errores de negocio dentro de un controlador, la traducción ya está en dos

sitios y uno de los dos acabará envejeciendo. La correspondencia entre causa y código se detalla más adelante, en el capítulo de comunicación.

Este proyecto es además el único de la solución que genera archivo de documentación, porque la página de documentación autogenerada de la interfaz de programación lee de ahí las descripciones de cada operación y de ningún otro sitio. Sin ese archivo, la página listaría rutas y códigos sin decir qué significan.

Los adaptadores salientes

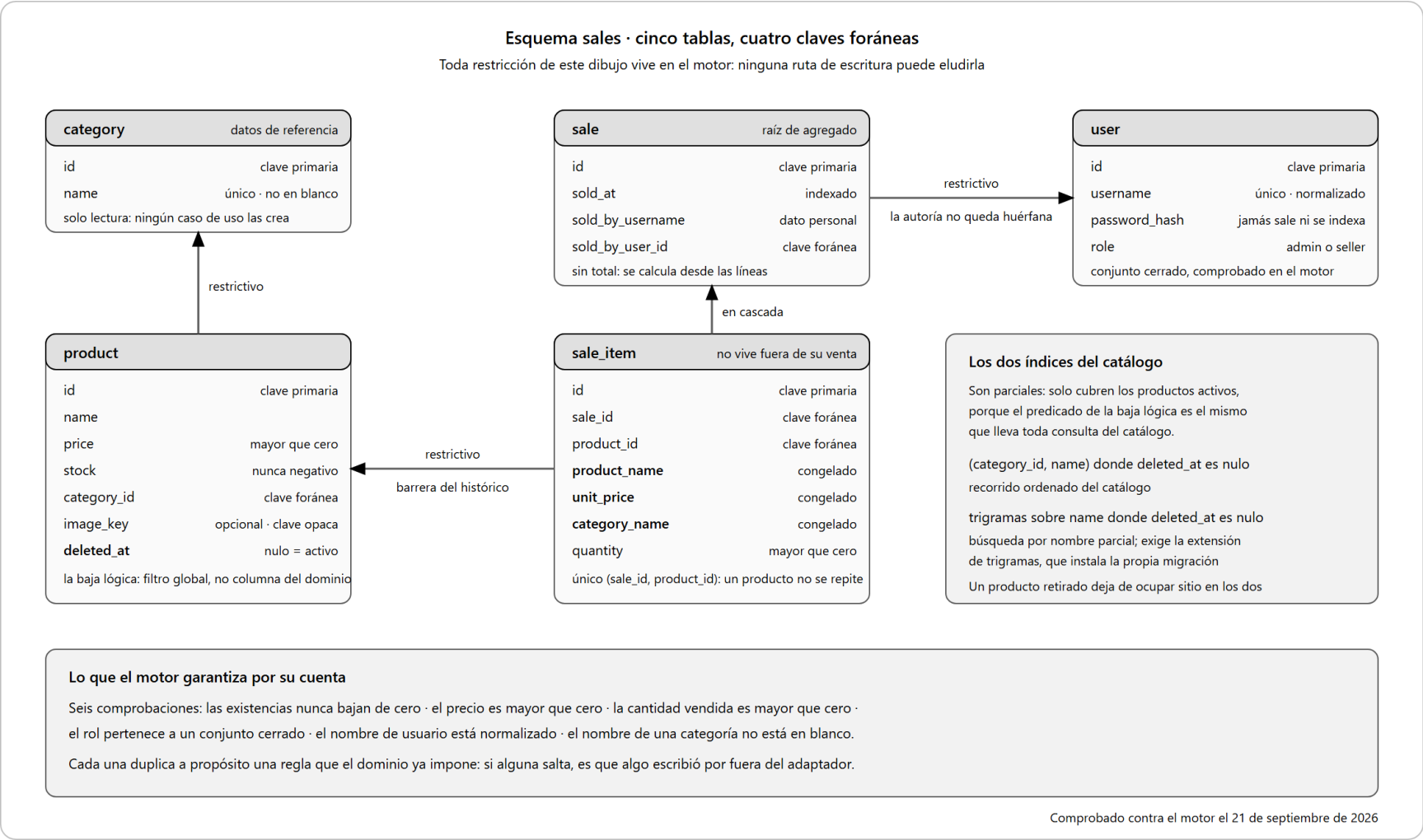
Persistencia

Es el adaptador más extenso. Contiene el contexto de datos, cinco configuraciones de mapeo —una por entidad persistida—, cuatro repositorios, la consulta agregada del reporte, la unidad de trabajo y el conjunto de migraciones. El mapeo se declara por configuración explícita y no por convención, de modo que ninguna decisión de esquema queda implícita.

El esquema sostiene garantías que no están en el código sino en el motor, y que por tanto ninguna ruta de escritura puede eludir (véase la Figura 7). Comprobadas contra el motor en ejecución el 21 de septiembre de 2026, son **cuatro claves foráneas** —la del producto a su categoría, la de la línea a su venta en cascada, la de la línea a su producto en modo restrictivo como barrera del histórico, y la de la venta a su autor, también restrictiva, para que la autoría de una venta no quede huérfana— y **seis comprobaciones de valor**: las existencias no bajan de cero, el precio es mayor que cero, la cantidad vendida es mayor que cero, el rol pertenece a un conjunto cerrado, el nombre de usuario está normalizado y el nombre de una categoría no está en blanco.

Figura 7

El modelo de datos: cinco tablas, cuatro claves foráneas y lo que el motor garantiza por su cuenta



Nota. Los atributos en negrita de `sale_item` son los que la línea congela al vender, y `deleted_at` es la marca de baja lógica del producto. Elaboración propia a partir de una consulta al catálogo del motor `simple-stock-flow-db-1` el 21 de septiembre de 2026.

Cada una de esas comprobaciones duplica a propósito una regla que el dominio ya impone. La duplicación no es redundancia ociosa: si alguna salta, significa que algo escribió por fuera del adaptador, y debe verse en lugar de capturarse en silencio.

Los dos índices del catálogo son **parciales** y solo cubren los productos activos —uno ordena por categoría y nombre, el otro es de trigramas sobre el nombre—, lo que exige instalar una extensión del motor durante la migración y, con ella, que el usuario de la base pueda crearla. El predicado del índice es el mismo que lleva toda consulta del catálogo, que es la condición para que el motor llegue a usarlo.

Las nueve migraciones (véase la Tabla 4) se aplican en orden al arrancar el servicio, y cada una explica una porción del esquema anterior.

Tabla 4

Migraciones del esquema, en orden de aplicación

Migración	Qué introduce
InitialSchema	El esquema completo y las categorías de referencia como datos sembrados
StockNonNegative	La comprobación de motor que impide unas existencias negativas
AccentSeedCategoryNames	La corrección de los nombres acentuados de las categorías sembradas
RenameTablesToSingular	La unificación de los nombres de tabla en singular
EngineInvariants	Las comprobaciones adicionales exigidas al motor: precio, cantidad, rol, nombre de usuario y nombre de categoría
ProductLogicalDeletion	La columna y el filtro que sostienen la baja lógica
FreezeCategoryOnSaleItem	La columna que congela la categoría en la línea de venta, obligatoria y sin valor por omisión
SaleAuthorWithReferentialIntegrity	La clave foránea de la venta hacia su autor, en modo restrictivo, con su índice
CatalogueAccessIndexes	La extensión de trigramas y los dos índices parciales del catálogo, que solo cubren los productos activos

Nota. Las categorías de referencia viajan dentro de la migración inicial y no en un guion de siembra aparte. Son nueve migraciones, más la instantánea del modelo que el mapeador mantiene. Verificado contra el motor el 21 de septiembre de 2026.

Seguridad

Implementa dos puertos. `JwtTokenGenerator` emite el testigo de sesión en el formato definido por la norma correspondiente (Jones et al., 2015), con validación de emisor, audiencia, vigencia y firma, y una tolerancia de reloj acotada. `Pbkdf2PasswordHasher` deriva y verifica contraseñas con la función de derivación basada en contraseña especificada por Moriarty et al. (2017).

La clave de firma no tiene valor por omisión y se exige de al menos treinta y dos caracteres: si falta o es demasiado corta, **la interfaz de programación se niega a arrancar** y explica por qué. Arrancar con una clave de relleno habría funcionado en apariencia y habría repartido testigos falsificables, con un fallo que aparecería lejos de su causa.

Almacenamiento de binarios

`LocalStorage` guarda las imágenes de producto en un volumen, nunca dentro de la imagen del contenedor, y devuelve una **clave opaca** —un identificador de treinta y dos caracteres sin relación con el nombre original del archivo—. El dominio nunca ve rutas ni bytes: solo esa clave. Sustituir el disco local por un almacén de objetos es, por tanto, cambiar una implementación de puerto y ninguna otra cosa.

El borrado de un binario tiene un orden obligatorio y una promesa que no se hace. Primero se anula la clave en la base y se confirma la transacción; **después** se borra el binario. El almacenamiento no participa en la transacción del motor, de modo que la atomicidad entre ambos pasos no es alcanzable y no se promete: un binario huérfano es inofensivo, mientras que una clave que apunta a un binario borrado es una imagen rota permanente.

La raíz de composición y el arranque

`bootstrap/Composition/PortBindings.cs` es **el** punto de composición del hexágono: el único archivo del repositorio donde un puerto encuentra su adaptador. La regla que impone se

enuncia y se verifica con facilidad: una construcción directa de infraestructura fuera de ese archivo significa que el hexágono está roto.

El resto del proyecto de arranque reúne `HostConfiguration`, que agrupa lo que es transporte y no hexágono —autenticación, política de origen cruzado, documentación autogenerada, aplicación de migraciones y servicio de archivos estáticos—;

`AdministratorBootstrap`, que crea el primer administrador a partir de variables de entorno porque una base recién creada no tiene a nadie que pueda emitir un testigo; `SystemClock`, la implementación del reloj; `SwaggerDocumentation`, con los filtros que ajustan la página generada; y `Program.cs`, que queda reducido a una lista legible de decisiones.

Los artefactos de prueba

Las pruebas se organizan en tres niveles, y la separación en proyectos distintos es lo que impide que un nivel se contamine con las herramientas del siguiente.

- **Dominio.** Referencia solo al proyecto de dominio y **no incluye ninguna biblioteca de dobles de prueba**. El comentario del archivo de proyecto lo dice sin rodeos: el día que haga falta un simulacro ahí dentro, el diseño está mal.
- **Aplicación.** Sustituye los puertos salientes por dobles y no referencia ningún adaptador. Cubre, entre otras cosas, el reintento por conflicto de concurrencia —contando los intentos— y la superficie de escritura de la venta.
- **Adaptadores.** Levanta el proceso completo con el servidor de pruebas del marco web y lo ejecuta contra un **motor PostgreSQL real** administrado por contenedores efímeros. Comprueba el esquema inicial, las invariantes del motor, la concurrencia de inventario, la forma de los reclamos del testigo, la traducción de errores, el

almacenamiento de archivos y el contrato de los extremos de catálogo, ventas y reportes.

El artefacto de imagen

El `Dockerfile` del servicio es de varias etapas: restaura dependencias con los manifiestos primero para que la restauración permanezca en caché mientras no cambien, publica en una etapa con el kit de desarrollo y copia el resultado a una imagen de solo ejecución. Los proyectos de prueba quedan fuera del contexto de construcción a propósito. El proceso se ejecuta con un usuario sin privilegios, y el directorio de binarios se crea y se cede a ese usuario antes de cambiar de identidad.

Artefactos del portal

El portal es una aplicación Angular 20 con componentes autónomos y carga diferida por ruta. Su estructura de directorios repite la del servicio, y esa simetría no es decorativa: es lo que permite probar la política del carrito sin levantar un servidor.

El dominio del portal

`domain/` no importa nada de Angular. Contiene los modelos `product`, `sale`, `session` y `paged-result`; los objetos de valor `money`, `quantity` y `date-range`; y dos políticas con nombre propio: la del carrito y la de existencias. Que el portal tenga políticas propias no duplica las reglas del servidor: el servidor sigue siendo la autoridad, y el portal se limita a no dejar que la persona construya una petición que ya se sabe inválida.

La capa de aplicación del portal

Declara cuatro puertos —repositorio de productos, de ventas, de reportes y de autenticación, este último acompañado de un puerto de almacenamiento de sesión— y nueve casos de uso: iniciar sesión, listar productos, obtener un producto, guardar un producto, listar

categorías, registrar una venta, listar ventas, obtener el reporte y dar de alta un vendedor. El estado compartido vive en tres almacenes reactivos: el del carrito, el del catálogo y el de la sesión.

Ningún caso de uso conoce el cliente de transporte. Cambiar el transporte —o sustituirlo por dobles para una prueba— consiste en cambiar una lista de proveedores.

La capa de infraestructura del portal

Implementa los puertos con cuatro repositorios sobre el cliente de transporte del marco, un almacenamiento de sesión sobre el almacenamiento local del navegador, tres traductores entre las declaraciones de tipos del transporte y los modelos de dominio, y dos interceptores.

El interceptor de autenticación es un artefacto pequeño con una consecuencia grande: **si hay sesión, añade la cabecera de autorización a todo lo que sale**, de modo que ningún caso de uso sabe que existe una cabecera. El interceptor de errores traduce la respuesta del servidor a un mensaje legible. `infrastructure/providers.ts` es la raíz de composición del portal, el equivalente exacto del archivo de enlaces del servicio.

La capa de presentación

`presentation/` agrupa las pantallas por dominio funcional —autenticación, productos, ventas, reportes y vendedores—, dos guardas de ruta, la envoltura común de la aplicación y las piezas de interfaz compartidas. La tabla de rutas declara la carga diferida de cada pantalla y aplica las guardas: todo cuelga de una ruta protegida por la guarda de sesión, y tres rutas exigen además la guarda de administrador —el alta y la edición de producto, y el alta de vendedor—. Las rutas están en español, conforme a la frontera idiomática del sistema, mientras que los identificadores de los componentes están en inglés.

Artefactos de despliegue del portal

`nginx.conf` es un artefacto de arquitectura por derecho propio, porque es donde se decide qué llega al servicio y qué no. Declara cinco bloques de localización: tres de ellos —el de binarios, el de la página de documentación y el de la interfaz de programación— usan la forma de prefijo con precedencia, porque el servidor evalúa las expresiones regulares antes que los prefijos y el bloque de recursos estáticos se quedaba con las imágenes, con los recursos de la página generada y con cualquier ruta del servicio terminada en una extensión de recurso estático. Resuelve además la dirección del servicio de forma dinámica, porque el motor de contenedores asigna una dirección nueva cada vez que un contenedor se recrea, y eleva el límite de tamaño del cuerpo para que sea el servicio —y no el intermediario— quien rechace un archivo demasiado grande con un mensaje que una persona pueda leer.

`proxy.conf.json` cumple el mismo papel durante el desarrollo con el servidor de desarrollo del marco, apuntando al puerto que publica el solapamiento de desarrollo. El `Dockerfile` del portal construye el paquete con Node y lo sirve desde una imagen de servidor web, copiando la configuración anterior.

Artefactos de infraestructura

La composición de contenedores

`docker-compose.yml` declara tres servicios, dos volúmenes con nombre y una red implícita. Cada servicio tiene comprobación de salud propia, y las dependencias están expresadas por **condición de salud** y no por orden de arranque. El único puerto publicado es el del portal.

`docker-compose.dev.yml` es un solapamiento que publica lo que el desarrollo necesita y la producción no debe exponer: la base, para poder inspeccionarla con un cliente, y la interfaz de programación, para que el servidor de desarrollo del portal tenga con quién hablar y para que las sondas documentadas contra ese puerto sean reproducibles. Mezclar ambos arranques recrea el

contenedor del servicio y le retira el puerto publicado sin avisar, que es la causa más habitual de que ese puerto deje de responder de golpe.

El orden de arranque no lo marca una lista de precedencias sino las comprobaciones de salud: el motor arranca primero, el servicio espera a que el motor responda —no a que su contenedor exista— y aplica entonces las migraciones y la siembra del primer administrador, y el portal espera a que el servicio responda. De ahí que no haya un primer arranque que falla y un segundo que funciona.

La configuración por entorno

`.env.example` declara todas las claves de configuración y ningún valor secreto (véase la Tabla 5); el archivo real queda fuera del control de versiones. Es, además, el artefacto que documenta dónde están los datos.

Tabla 5

Claves de configuración declaradas en el archivo de ejemplo

Clave	Para qué sirve
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Lo único que necesita un cliente para conectarse al motor
JWT_SIGNING_KEY	Firma del testigo de sesión. Sin valor por omisión y con longitud mínima: si falta, el servicio no arranca
ADMIN_USERNAME, ADMIN_PASSWORD	Credenciales del primer administrador, creado en el primer arranque. Sin él no hay forma de entrar
PORTAL_PORT	El único puerto que el sistema expone al anfitrión
SWAGGER_ENABLED	Monta o retira la página de documentación de la interfaz de programación
DEMO_SELLER_PASSWORD	Cuenta de vendedor que crea el sembrador de demostración, para poder ver en pantalla lo que un vendedor no puede hacer

Nota. Ninguna clave sensible tiene valor por omisión. Un valor de relleno habría permitido arrancar un sistema aparentemente sano con una garantía que no existe.

El guion de verificación

`verify.sh` es el artefacto que sostiene la afirmación de que la infraestructura funciona, y se escribió **antes** que el archivo de composición al que juzga. Comprueba, entre otras cosas, que el portal responde, que un extremo protegido contesta con el código de no autenticado —lo que demuestra a la vez que el intermediario alcanza a la interfaz de programación y que esta guarda sus extremos—, que los datos sobreviven a un reinicio, que el servicio se niega a arrancar sin clave de firma, que las imágenes atraviesan el intermediario y que la página de documentación es alcanzable por las dos rutas.

Una decisión suya merece destacarse: el guion **reconstruye las imágenes** antes de medir, porque adoptar la imagen que ya lleva la etiqueta deja servir un paquete anterior mientras todas las comprobaciones siguen en verde. Es un caso concreto de una suite que pasa sobre un sistema que no contiene los cambios que se creen desplegados.

Los artefactos de demostración

El repositorio `simple-stock-flow-tools` contiene en su raíz el catálogo de ejemplo, el sembrador y el generador de láminas, y en el subdirectorio `img/` las veintidós imágenes de producto. El sembrador crea todo **por la interfaz de programación y no por sentencias directas contra la base**: los datos de demostración obedecen exactamente las mismas reglas que lo que teclea una persona, de modo que un precio que rompe una invariante falla ahí igual que fallaría en pantalla. La excepción es una y está declarada, y conviene decir en qué consiste, porque es una sentencia: un `update` sobre `sales.sale`, aislado en la función `spread_sales_over_time` y ejecutado cuando cada venta ya pasó por todas las reglas. Existe porque el dominio sella la venta con la hora actual —que es lo correcto—, y eso dejaría todas las ventas de un sistema recién sembrado dentro del mismo minuto, sin nada que mostrar en el reporte por rango de fechas. Ese `update` es lo único que el repositorio **ejecuta** contra la base.

Hay un segundo bloque de sentencias directas —cuatro borrados que vacían las tablas y conservan las categorías y el administrador—, pero el sembrador no lo ejecuta: lo compone y lo imprime cuando encuentra el catálogo ocupado, para que lo pegue quien decida empezar de cero.

Que ese material viva aparte, y no dentro de la unidad de orquestación, obedece a un criterio que merece enunciarse porque es reutilizable: **una herramienta opcional no debe imponer sus dependencias al artefacto que la aloja**. La infraestructura declara que para levantar el sistema basta con el motor de contenedores, mientras que las herramientas exigen además un intérprete de Python y, sobre él, requisitos que cambian de un guion a otro: el sembrador necesita la biblioteca de peticiones web y el propio motor de contenedores —el único requisito que comparte con la infraestructura—, porque reparte las ventas en el tiempo a través de él; el generador de láminas necesita la biblioteca de imágenes y un navegador. Ni el archivo de composición ni el guion de verificación referenciaban esas herramientas, de modo que la separación no desconectó nada: solo le devolvió a la unidad de orquestación la promesa que ya hacía.

Volúmenes y red

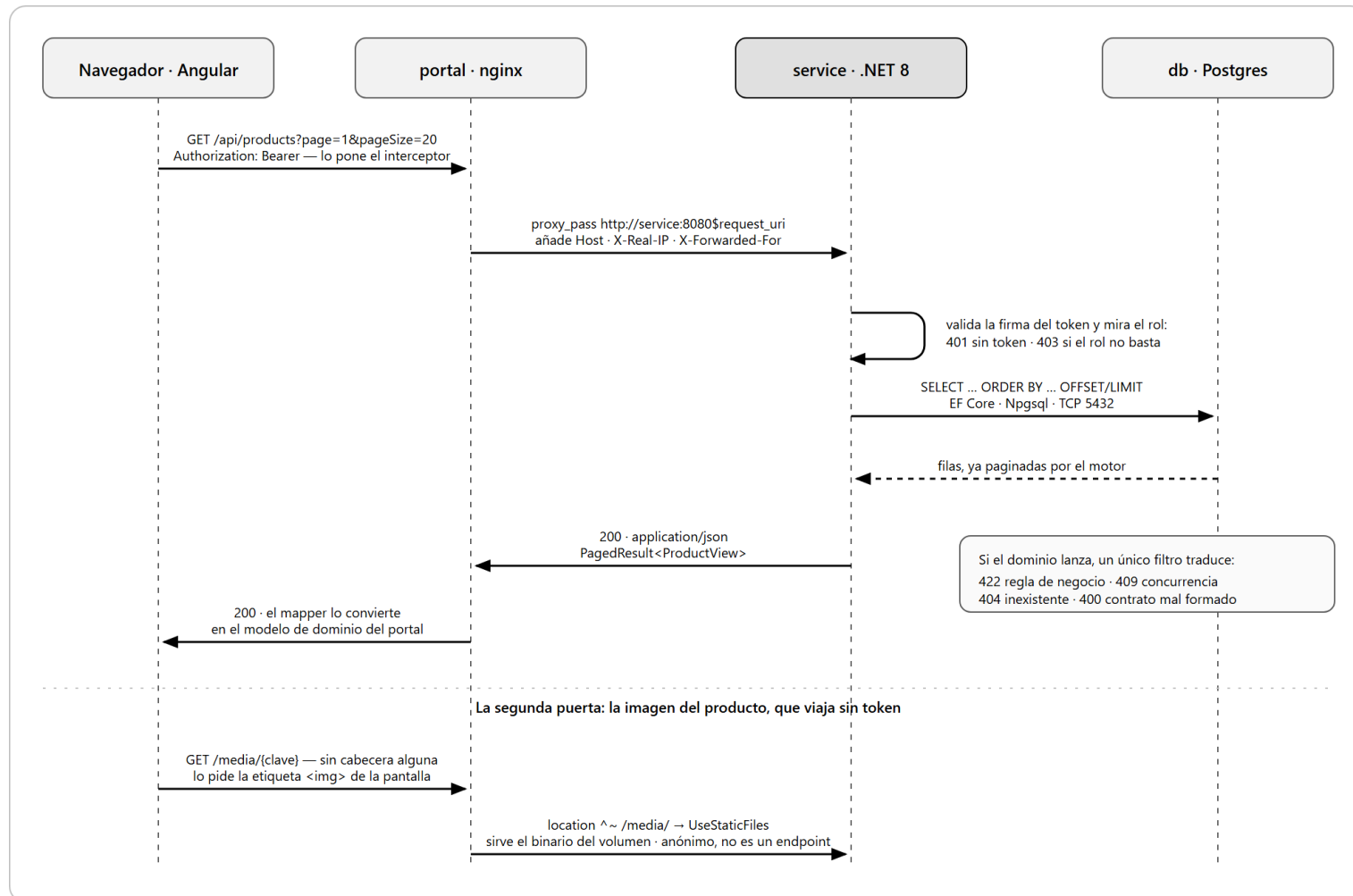
Dos volúmenes con nombre sostienen el estado: uno para los datos del motor y otro para los binarios subidos. Los binarios viven en un volumen y nunca dentro de la imagen, que es lo que permite reconstruir el contenedor sin perderlos. La red es privada, y el nombre de servicio del contenedor de la interfaz de programación forma parte del contrato: el intermediario del portal reenvía a ese nombre, de modo que renombrarlo rompe la interfaz de usuario.

La comunicación entre las unidades

Descritas las piezas, queda lo que las une. Este capítulo recorre una petición completa (véase la Figura 8) y enumera, frontera por frontera, qué protocolo la cruza, quién la autoriza y qué formato transporta.

Figura 8

Camino completo de una petición, de la pantalla al motor



Nota. La parte superior corresponde a una consulta del catálogo; la inferior, a la segunda puerta del sistema, la de los binarios, que viaja sin testigo. Elaboración propia a partir de `nginx.conf`, del adaptador entrante y del código de arranque del servicio.

Fronteras y protocolos

La Tabla 6 recorre las cinco fronteras del sistema. La última existe solo en desarrollo, y distinguirla importa: un despliegue con el archivo base no expone la interfaz de programación.

Tabla 6

Qué cruza cada frontera del sistema

Tramo	Cómo viaja	Quién autoriza	Qué transporta
Navegador a portal	Protocolo web por el único puerto publicado	Nadie: es contenido estático	El paquete de la aplicación y, después, cada llamada
portal a service	Reenvío por nombre de servicio dentro de la red privada	El servicio, con el testigo que le llega íntegro	Notación de objetos JavaScript
service a db	Conexión con el motor a través del proveedor de datos	La cadena de conexión del entorno	Consultas, con la agregación del reporte resuelta en el motor
service a volumen	Sistema de archivos del contenedor	No aplica	Los bytes de las imágenes
Anfitrión a service	Protocolo web contra el puerto que publica el solapamiento de desarrollo	El mismo testigo que cualquier otro cliente	Las sondas de verificación, desde <i>infra</i> , y el sembrado de demostración, desde <i>tools</i>

Nota. El tramo del anfitrión al servicio solo existe cuando se aplica el solapamiento de desarrollo; en un despliegue con el archivo base, la interfaz de programación no es alcanzable desde fuera de la red.

Autenticación y autorización

La sesión se abre contra el extremo de autenticación y devuelve un testigo firmado. A partir de ahí, el interceptor del portal lo adjunta a cada petición, el intermediario lo transporta sin tocarlo y el servicio lo valida: firma, emisor, audiencia y vigencia. La ausencia de testigo produce el código de no autenticado; un testigo válido con un rol insuficiente produce el código de prohibido. Que ambos códigos sean distinguibles es un criterio de aceptación explícito y no un detalle de implementación: confundirlos hace indistinguible «no sé quién eres» de «sé quién eres y no puedes».

El mantenimiento del catálogo exige rol de administrador; consultar productos, registrar ventas y consultar el reporte solo exigen sesión. El alta de usuarios exige administrador, lo que crea el problema del primer arranque —no hay nadie que pueda emitir un testigo— y que resuelve el artefacto de arranque descrito antes.

La ruta pública de binarios

La imagen de un producto se sirve bajo una ruta que **no es un extremo enrutado**: la atiende el middleware de archivos estáticos sobre el volumen de binarios, bajo la misma dirección que publica el adaptador de almacenamiento. Al no estar enrutada, no atraviesa la autorización, y no hay forma de protegerla sin cambiar el mecanismo.

La consecuencia está medida y aceptada: las claves son opacas y no se pueden adivinar ni enumerar, pero una dirección filtrada sirve la imagen a cualquiera de forma indefinida. Es lo que hace que la etiqueta de imagen del portal funcione sin cabeceras, y figura como riesgo aceptado con dueño en el capítulo correspondiente. Documentarlo es la diferencia entre un riesgo asumido y un descuido.

La traducción de errores

Toda excepción de dominio se convierte en respuesta en un solo lugar (véase la Tabla 7). La correspondencia distingue explícitamente el caso en que la petición estaba bien formada pero la regla de negocio falló, que es el que con más frecuencia se confunde con un error de contrato (Fielding & Reschke, 2014).

Tabla 7*Correspondencia entre causa y código de respuesta*

Origen	Código	Por qué ese y no otro
Violación de una invariante de negocio	422	La petición estaba bien formada; lo que falló fue la regla
Contrato mal formado	400	El cuerpo o los parámetros no cumplen la forma declarada. La respuesta nombra los campos en español y añade un identificador de traza
Sin testigo o testigo inválido	401	Es un problema de autenticación
Rol insuficiente	403	Es un problema de autorización, y debe ser distinguible del anterior
Recurso inexistente	404	No hay nada que identifique ese recurso
Conflicto de concurrencia	409	Se agotaron los tres reintentos acotados que fija la Decisión 2

Nota. El cuerpo del error no es uniforme: el filtro produce una de sus formas y el marco web produce las otras. La forma que produce el marco arrastra una limitación conocida, recogida en el registro de riesgos.

Decisiones de arquitectura

Las cuatro decisiones que siguen son las que fijaron la forma del sistema. Cada una cerró una pregunta que admitía varias respuestas razonables, y cada una impuso una consecuencia que el resto del diseño tuvo que aceptar. Se registran con el formato propuesto por Nygard (2011) — contexto, alternativas consideradas, decisión y consecuencias— por un motivo práctico: **una decisión sin sus alternativas descartadas no se puede revisar, solo se puede creer**. Quien vuelva sobre una de ellas dentro de un año necesita saber qué se estudió y por qué se rechazó, no solo qué se eligió.

La Tabla 8 las resume con la alternativa principal que cada una descartó; los apartados siguientes desarrollan las catorce opciones que se estudiaron y el motivo concreto de cada rechazo.

Tabla 8

Las cuatro decisiones de arquitectura, con la alternativa principal que cada una descartó

Decisión	Qué se decidió	Alternativa principal descartada, y por qué	Consecuencia que impone
1. La propiedad del esquema	El esquema pertenece al servicio; la infraestructura entrega un motor vacío	Crear el esquema con guiones de inicialización del contenedor del motor: solo se ejecutan con el volumen vacío, de modo que la segunda versión no se aplicaría jamás y el fallo sería silencioso	Las migraciones se aplican al arrancar el servicio, lo que obliga a una sola instancia mientras no se muevan a un paso previo del despliegue
2. El control de concurrencia	Control optimista sobre el testigo de versión que el propio motor mantiene por fila	Bloqueo pesimista al leer los productos: serializaría ventas que no comparten ningún producto, y filtraría el bloqueo al contrato de un puerto cuya firma dice que solo lee	El conflicto se reintenta tres veces con espera irregular y, agotados los intentos, se responde 409
3. La retirada de un producto	Baja lógica con marca de fecha, declarada fuera del agregado y con filtro global de consulta	Borrado físico con clave foránea restrictiva: la operación de borrado fallaría justo para los productos que se vendieron, con un error que quien opera no puede resolver	Un producto retirado desaparece de toda consulta por omisión; olvidar el filtro deja de ser posible
4. El reporte agregado	El reporte agrega en el motor y opera sobre los atributos que la línea de venta congeló	Unir contra el catálogo vivo para resolver el nombre y la categoría: recategorizar o renombrar un producto reescribiría reportes de períodos ya cerrados	La línea de venta guarda nombre, precio y categoría del momento de la venta, y no sigue al catálogo

Nota. Las cuatro son autónomas: cada una se entiende sin leer las otras tres.

Decisión 1 · La propiedad del esquema

Contexto. Alguien tiene que crear las tablas, los índices y los datos de referencia, y había dos sitios plausibles para esa responsabilidad: la unidad de infraestructura, junto al contenedor del motor, o la del servicio, junto al modelo que las usa. La pregunta de fondo no era de comodidad sino de rastreabilidad: **qué cambio explica el estado del esquema.**

Decisión. Las migraciones se derivan del modelo de persistencia, viven dentro del adaptador de persistencia del servicio y se aplican al arrancar. La unidad de infraestructura

levanta el motor, la base vacía, el usuario, la red y el volumen, y no contiene una sola sentencia de definición de datos.

Alternativas consideradas. Se estudiaron tres y se rechazaron las tres.

- **Definición de datos versionada a mano, en pares de subida y bajada,** ejecutada por un lanzador independiente desde la unidad de infraestructura. Es la opción correcta cuando varios servicios comparten la misma base o cuando quien administra el motor no es quien desarrolla. Aquí hay **un solo consumidor**, y el coste real habría sido mantener a mano una definición que el modelo ya deriva, con la garantía añadida de que ambas divergieran.
- **El esquema creado por guiones de inicialización del contenedor del motor.** Solo se ejecutan cuando el volumen está vacío: la segunda versión del esquema no se aplicaría nunca, y el fallo sería silencioso. Un fallo silencioso en el esquema es el peor de los disponibles, porque el sistema sigue respondiendo.
- **Sincronización automática del modelo, sin migraciones.** No deja historia, no es reversible y no se puede revisar en un cambio. Ahorra el paso de generar la migración y a cambio elimina el único artefacto que explica por qué el esquema es como es.

Consecuencias. A favor: el cambio que modifica una entidad trae consigo la migración que lo soporta, y no se pueden separar; la infraestructura es sustituible, porque cualquier motor accesible sirve, incluido uno gestionado; y un entorno limpio queda completo ejecutando únicamente el servicio. En contra, y declarado: aplicar migraciones en el arranque **no es seguro con varias réplicas**, de modo que el sistema queda atado a una sola instancia mientras no se mueva ese paso; el arranque del servicio depende de que el motor esté listo y no solo levantado,

lo que exige espera activa; y quien revise el esquema tiene que leer una migración generada en lugar de un archivo escrito a mano.

Decisión 2 · El control de concurrencia sobre las existencias

Contexto. El agregado de producto garantiza que las existencias no queden negativas **en memoria**. Entre que el caso de uso lee los productos y confirma la transacción hay una ventana en la que otra venta puede retirar del mismo producto: ambas leen las mismas existencias, ambas pasan la invariante y la base queda en negativo aunque el dominio se haya cumplido en las dos. El criterio CA-04.6 declara ese resultado inaceptable: dos ventas simultáneas del último ejemplar deben resolverse con una confirmación y un conflicto, no con dos confirmaciones.

Decisión. Control optimista con el testigo de versión que el propio motor mantiene por fila, expuesto como propiedad sombra para que el dominio no lo vea. Toda actualización condiciona la escritura a que la fila no haya cambiado desde que se leyó; si cambió, ninguna fila resulta afectada y el conflicto se detecta. Ante conflicto se reintenta **el caso de uso completo desde la lectura**, tres veces con espera irregular, y agotados los intentos se responde con conflicto. La comprobación de existencias no negativas queda en el motor como última barrera, y la excepción de concurrencia del mapeador no sale del adaptador: es un tipo de infraestructura, y verlo en la capa de aplicación significaría que la regla de dependencia está rota.

Alternativas consideradas. Se estudiaron cuatro.

- **Una columna de versión mantenida por la aplicación.** Exige que **todas** las rutas de escritura la incrementen. El día que una migración, una semilla o un adaptador nuevo la olvide, el mecanismo queda desactivado en silencio y nadie se entera hasta que hay existencias negativas.

- **Bloqueo pesimista al leer los productos.** El caso de uso lee varios productos por identificador: bloquearlos durante toda la venta **serializa ventas que no comparten ningún producto**. Además convertiría un puerto de lectura en una lectura que bloquea, algo que su firma no expresa, de modo que el bloqueo se filtraría al contrato.
- **Aislamiento serializable.** Convierte el mismo conflicto en un fallo de serialización que también hay que reintentar —mismo coste— pero imponiendo bloqueo de predicados a **todas** las lecturas, incluidas las del reporte, que no tienen nada que evitar.
- **Decremento condicional en una sola sentencia.** Es correcto y más barato, pero **mueve la decisión de negocio al lenguaje de consulta**: el dominio dejaría de ser la autoridad sobre su propia invariante. Se descarta por frontera y no por rendimiento, y por eso queda escrito aquí como **la vía de salida si el reintento resultara caro**: una decisión con salida escrita vale más que una decisión cerrada.

Consecuencias. A favor: la superficie de esquema es **cero**, porque no hay columna que el dominio pueda exponer por accidente, y el motor mantiene el testigo en toda actualización sin depender de la disciplina de quien programa. En contra, y declarado: el testigo cambia ante **cualquier** actualización de la fila, incluido un renombrado o un cambio de imagen, lo que produce conflictos que una versión ligada solo a las existencias no produciría —el reintento los absorbe—; el mecanismo es específico del motor elegido, y con otro el equivalente sería una columna de versión nativa; y reintentar solo la escritura en lugar del caso de uso completo reaplicaría un descuento calculado sobre existencias viejas, que es el error más probable al implementarlo.

Decisión 3 · La retirada de un producto del catálogo

Contexto. El sistema debe permitir retirar un producto. Pero la línea de venta referencia al producto y el reporte histórico depende de que esa fila siga existiendo. Borrar físicamente un producto vendido rompe el histórico; impedir el borrado deja a quien administra con un botón que falla justo para los productos que importan: los que se vendieron.

Decisión. Baja lógica con una marca de fecha, nula mientras el producto está activo. La marca **no existe en el agregado**: se declara como propiedad sombra en la configuración de persistencia, con filtro global de consulta. La operación de baja pasa a ser una actualización, el borrado físico desaparece del camino de la aplicación, y la clave foránea de la línea hacia el producto queda como barrera de última instancia para que un borrado manual falle ruidosamente.

El contrato de los repositorios es explícito y tiene tres reglas, porque las tres son distintas: la búsqueda del catálogo **filtra** las bajas, porque un producto retirado no se ofrece; la carga de varios productos por identificador previa a vender **también filtra**, y sin esa regla se podría vender un producto retirado; y la carga de uno solo por identificador **no filtra**, porque una línea de venta histórica tiene que poder resolver su producto.

Alternativas consideradas. Se estudiaron tres.

- **Borrado físico con clave foránea restrictiva.** La operación de borrado fallaría exactamente para los productos con ventas, y quien opera recibiría un error que no puede resolver por sí mismo. Un sistema que ofrece un botón que falla de forma predecible ofrece un botón roto.
- **Una marca booleana en lugar de una fecha.** El booleano no responde *cuándo* se dio de baja, y se indexa mal: dos valores efectivos. La fecha nula, en cambio, funciona como predicado de índice parcial, que es exactamente lo que los dos índices del catálogo aprovechan.

- **Una propiedad real en el agregado.** El dominio ganaría un concepto que es de persistencia. Es defendible verlo como regla de negocio —«un producto vendido se retira, no se elimina»—, pero el puerto ya expone una operación de borrado, y añadirlo al agregado tendría mayor radio de cambio sin beneficio observable.

Consecuencias. A favor: el dominio no se entera, y el filtro global hace que **lo seguro sea lo que ocurre por omisión**, de modo que la excepción tenga que escribirse a mano. La forma inversa —filtrar en cada consulta y confiar en no olvidarlo— es exactamente por donde se cuele la venta de un producto retirado. En contra, y declarado: las filas no se recuperan nunca y la tabla solo crece, lo que es irrelevante al volumen previsto; todo índice que sirva al catálogo debe llevar el predicado de baja, y toda consulta que quiera usarlo debe llevarlo literalmente o el motor no lo usará; y queda una regla para el futuro: **cualquier restricción de unicidad que se añada sobre el producto debe ser parcial sobre los activos**, porque de lo contrario un producto retirado reservaría su código para siempre.

Decisión 4 · El reporte agregado sobre atributos congelados

Contexto. El reporte de ventas por rango es el entregable funcional de más peso del sistema. Necesita, por producto: unidades, importe, nombre y categoría. Había dos problemas. El puerto de ventas existente devuelve agregados de venta completos para un rango y sin paginar, de modo que usarlo obliga a cargar todas las ventas del período en memoria, que es lo que el criterio CA-06.5 declara defecto. Y el nombre de la categoría **no era alcanzable** desde la venta: la línea congelaba el nombre del producto pero no el de su categoría, y el servicio del reporte no inyecta ningún repositorio de catálogo.

Decisión. Dos cosas, y las dos son necesarias. Primera: un puerto saliente de solo lectura propio del reporte, que devuelve las filas ya agregadas; el adaptador lo implementa con una

consulta que agrupa **en el motor**, y el servicio de aplicación no ve una sola entidad de venta.

Segunda: la línea de venta congela también el nombre de la categoría, igual que ya congelaba el nombre y el precio del producto.

Alternativas consideradas. Se estudiaron cuatro.

- **Unir contra el catálogo vivo para resolver la categoría.** Recategorizar un producto **reescribiría reportes ya emitidos** de períodos cerrados. Viola el criterio CA-06.4 y el requisito RNF-02, que son precisamente los que dan sentido al reporte.
- **Inyectar repositorios de catálogo en el servicio del reporte.** Le daría dos razones para cambiar y reintroduciría, por la puerta de atrás, la lectura viva que la alternativa anterior descarta.
- **Guardar también el identificador de categoría en la línea.** Obligaría a unir contra la categoría para mostrar el nombre, que es de nuevo la lectura viva que se quiere evitar. El reporte agrupa por producto, no por categoría.
- **Una tabla de reporte precalculada.** Un agregado almacenado que puede desincronizarse, para un volumen que no lo necesita. Añade una fuente de verdad más sin resolver ninguna pregunta abierta.

Consecuencias. A favor: el reporte de un período cerrado es **estable por construcción y no por disciplina**; la consulta toca dos tablas en lugar de cuatro; y el servicio del reporte mantiene una sola responsabilidad. En contra, y declarado: cambia la firma de un método del agregado, porque añadir una línea pasa a requerir la categoría además del producto, y el caso de uso de venta gana una dependencia para resolverla —el coste no desaparece, se mueve del camino de lectura al de escritura, que es donde el dato se conoce—; y el nombre de la categoría se repite en cada línea, lo que **no es desnormalización**: dos líneas del mismo producto pueden

legítimamente llevar categorías distintas si se recategorizó entre ambas ventas, luego no existe la dependencia funcional que haría transitiva la repetición.

Atributos de calidad: qué se persigue y qué se paga

Un documento que enumera virtudes sin precio no describe una arquitectura: la anuncia. Los cinco apartados siguientes tienen todos la misma forma —**objetivo, mecanismo, cómo se verifica y qué se sacrifica**— porque el cuarto punto es el que distingue una decisión de una aspiración. Un atributo de calidad que no cuesta nada no se eligió: se supuso.

Fiabilidad e integridad

Objetivo. Ninguna ruta de escritura puede dejar el sistema en un estado que las reglas prohíben, ni siquiera bajo concurrencia. En concreto, `RN-01`: las existencias de un producto nunca son negativas, y `RNF-01`: dos ventas simultáneas del último ejemplar no dejan existencias negativas.

Mecanismo. La invariante se comprueba en el dominio al retirar existencias, y **además** el motor la sostiene con una comprobación propia, de modo que hay dos barreras y no una. Sobre esas dos se monta el control optimista de la Decisión 2: tres reintentos del caso de uso completo con espera irregular y, agotados, una respuesta de conflicto. El descuento de existencias y las líneas de la venta se confirman en una sola transacción: viajan juntos o no viaja ninguno.

Cómo se verifica. Con una prueba unitaria que ejerce el agregado; con una prueba de la capa de aplicación que fuerza el conflicto contra dobles y **cuenta los intentos**, para que el número no pueda cambiar sin que algo falle; y con una prueba de integración contra un motor PostgreSQL real que escribe por fuera del adaptador y comprueba que el motor lo rechaza.

Qué se sacrifica. Bajo carga concurrente sobre el mismo producto, **una venta legítima puede fallar** con un conflicto tras los tres intentos. Es una elección consciente: se prefiere una

negativa explícita, que quien vende puede resolver repitiendo la operación sobre unas existencias ya actualizadas, a un inventario negativo que nadie detecta hasta el recuento. El precio lo paga quien vende, en forma de reintento, y no el histórico.

Fidelidad histórica

Objetivo. RNF-02, verificado por el criterio CA-06.4: el reporte de un período cerrado no cambia porque el catálogo cambie. Un reporte que se mueve bajo los pies de quien lo leyó ayer no es un reporte, es una consulta.

Mecanismo. La línea de venta congela nombre, precio y categoría del momento de la venta; el motor exige esas columnas obligatorias y sin valor por omisión. No existe operación que modifique o anule una venta, ni en el agregado, ni en el puerto, ni en las rutas publicadas. Y la consulta del reporte **agrupa por los valores congelados** y no lee el catálogo vivo en ningún punto.

Cómo se verifica. Repitiendo el reporte sobre el mismo rango después de renombrar un producto, recategorizarlo y cambiarle el precio: devuelve exactamente lo mismo. Y comprobando, sobre la forma del sistema y no sobre una llamada, que no existe ninguna operación de modificación de venta en ninguna de las tres superficies.

Qué se sacrifica. Dos cosas, y las dos se notan en pantalla. Primera: un producto renombrado o recategorizado entre dos ventas del mismo rango aparece en **dos filas**, una por cada etiqueta que llegó a tener; sumadas dicen la verdad, pero la lectura exige una explicación que el manual de usuario tiene que dar. Segunda: una venta equivocada **no se corrige**, se compensa registrando la correcta. Ambos precios se pagan a cambio de que un período cerrado diga siempre lo mismo.

Seguridad y confidencialidad

Objetivo. RNF-08: las credenciales no se pueden recuperar del almacén, la sesión no se puede falsificar, y cada atributo persistido tiene una clasificación declarada y un manejo exigido —qué puede salir en una respuesta, qué puede aparecer en un registro de ejecución y qué no se borra nunca—.

Mecanismo. Las contraseñas se derivan con la función especificada por Moriarty et al. (2017) y nunca se almacenan en claro. El testigo de sesión se firma y se valida en emisor, audiencia, vigencia y firma. La clave de firma **no tiene valor por omisión**, se exige de al menos treinta y dos caracteres, y el servicio se niega a arrancar si falta. Los códigos de no autenticado y prohibido son distinguibles. El resultado de derivar una contraseña **jamás** sale en una respuesta, en un registro ni en una proyección, y no se indexa: su única lectura legítima es verificar, a través del puerto de derivación. El nombre de quien registra una venta es dato personal con acceso restringido y retención indefinida, y el reporte no lo expone: agrega por producto y no por operador.

Cómo se verifica. El arranque sin clave de firma falla y explica por qué; una prueba de integración inspecciona los reclamos del testigo emitido; y el guion de verificación mide que un extremo protegido responde con no autenticado cuando no hay testigo.

Qué se sacrifica. La ruta de los binarios. La imagen de un producto se sirve por el middleware de archivos estáticos y **no atraviesa la autorización**: una dirección filtrada sirve esa imagen a cualquiera de forma indefinida. La mitigación es que las claves son opacas y no se pueden enumerar, pero la mitigación no es una solución y así consta en el registro de riesgos. Se aceptó para que la etiqueta de imagen del portal funcione sin cabeceras; protegerla exige cambiar el mecanismo de servicio, no ajustar una política.

Mantenibilidad

Objetivo. Que una violación de la forma del sistema **falle al construir** y no en una revisión que alguien puede no hacer.

Mecanismo. Diez proyectos con referencias declaradas, de modo que la regla de dependencia sea una restricción del compilador. Un único punto de traducción de errores de dominio a códigos de respuesta. Un único archivo donde un puerto encuentra su adaptador. Y la propiedad `TreatWarningsAsErrors` activada para toda la solución, de modo que cualquier advertencia rompe la construcción.

Cómo se verifica. El grafo de referencias se lee en los archivos de proyecto sin ejecutar nada; una referencia que violara la regla no compila; una advertencia rompe la construcción; y una construcción directa de infraestructura fuera del archivo de composición es visible con una búsqueda de texto.

Qué se sacrifica. Diez proyectos para un problema que cabría en uno. El coste es real y no conviene disimularlo: más archivos, más tiempo de compilación, más ceremonia para añadir una clase y una curva de entrada más larga para quien llega. Se paga a cambio de que la regla la sostenga el compilador en lugar de la disciplina, que es lo único que sigue funcionando cuando quien escribió el sistema ya no está.

Rendimiento y escala

Objetivo. Que ninguna operación cargue en memoria un volumen que crece sin límite. Es `RNF-03` para las listas y el criterio `CA-06.5` para el reporte.

Mecanismo. El reporte agrega **en el motor** y devuelve una fila por producto; el servicio de aplicación no ve una sola entidad de venta, y el diseño del puerto impide escribirlo de otra manera. Toda lista que puede crecer viene paginada, con veinte elementos por omisión y cien como tope, aplicado recortando en lugar de rechazar, y con el tamaño servido —no el pedido—

en la respuesta. Los dos índices del catálogo son parciales sobre los productos activos, de modo que un producto retirado deja de ocupar sitio en ellos.

Cómo se verifica. El puerto del reporte devuelve filas agregadas y su firma no admite otra cosa; una prueba de contrato fija que el tamaño servido viaja en la respuesta; y el plan de la consulta usa los índices parciales porque la consulta lleva el mismo predicado.

Qué se sacrifica, y se declara. No hay objetivo de latencia ni de caudal, porque el enunciado del sistema no fija carga ni volumen esperados. Inventar una cifra para poder escribirla sería peor que no tenerla: nadie podría decir si se cumple. Y la Decisión 1 fija una sola instancia mientras las migraciones se apliquen durante el arranque, de modo que **el sistema no escala horizontalmente hoy**; la vía de salida —mover ese paso a un momento previo del despliegue— está escrita desde el primer día, y por eso es un límite y no una sorpresa.

Requisitos no funcionales y dónde se satisfacen

Un requisito no funcional que no señala el artefacto que lo cumple es una aspiración. La Tabla 9 enlaza los ocho con el mecanismo concreto que los realiza y con el lugar donde ese mecanismo vive.

Tabla 9*Realización de los requisitos no funcionales*

Código	Requisito	Mecanismo	Dónde vive
RNF-01	Integridad bajo concurrencia	Control optimista con tres reintentos y espera irregular; comprobación del motor como última barrera	Adaptador de persistencia; Decisión 2
RNF-02	Fidelidad histórica	Atributos congelados en la línea, agregación por esos valores y ausencia de operación de edición de venta	Dominio y consulta agregada; Decisión 4
RNF-03	Paginación	Tamaño por omisión de 20 y tope de 100, aplicado recortando; el tamaño servido viaja en la respuesta	Tipos comunes de la capa de aplicación
RNF-04	Arranque reproducible	Migraciones y datos de referencia aplicados al arrancar, encadenados por comprobaciones de salud	Arranque del servicio; Decisión 1
RNF-05	Trazabilidad	Registro estructurado de cada petición, con correlación y sin datos personales ni secretos	Adaptador entrante
RNF-06	Portabilidad del almacenamiento de binarios	Puerto con clave opaca; el dominio no ve rutas ni bytes	Capa de aplicación y adaptador de almacenamiento
RNF-07	Pruebas en tres niveles	Dominio sin dobles, aplicación con dobles, adaptadores contra motor real	Los tres proyectos de prueba
RNF-08	Confidencialidad y retención	Clasificación por atributo y manejo exigido: derivación de contraseña que no sale nunca ni se indexa, y ventas sin borrado ni edición	Adaptador de seguridad y configuración de persistencia

Nota. Cada fila es verificable de forma independiente; ninguna depende de la lectura de este documento. RNF-08 es el único requisito que no proviene del enunciado del sistema: se conserva a sabiendas porque el sistema guarda el nombre de quien vende.

Trazabilidad: de la regla al artefacto

Una decisión de arquitectura que no se puede atar a la exigencia que la obliga es una preferencia. La tabla de este capítulo recorre el camino completo en las dos direcciones: de la regla de negocio o el requisito hasta el artefacto que lo sostiene, y del artefacto de vuelta a la razón por la que existe.

Se incluyen únicamente las reglas y los requisitos con **consecuencia estructural**, es decir, aquellos cuyo cumplimiento cambió la forma del sistema y no solo el cuerpo de un método. La Tabla 10 los recoge.

Tabla 10

Trazabilidad de las reglas y los requisitos con consecuencia estructural

Regla o requisito	Qué obliga	Dónde se decide	Dónde vive el artefacto
RN-01	Las existencias de un producto nunca son negativas	Decisión 2	Agregado <code>Product</code> y comprobación <code>stock >= 0</code> del motor: dos barreras, no una
RN-05	Un producto no se repite dentro de una misma venta	—	Agregado <code>Sale</code> e índice único (<code>sale_id</code> , <code>product_id</code>) del motor
RN-06	El precio y el nombre de la línea quedan congelados al vender	Decisión 4	Entidad <code>SaleItem</code> , con las columnas obligatorias y sin valor por omisión
RN-07	Una venta registrada no se modifica ni se anula	Decisión 4	Una ausencia: no existe la operación en el agregado, ni en <code>ISaleRepository</code> , ni en las rutas de ventas
RN-08	Un producto vendido no se elimina: se retira del catálogo	Decisión 3	Marca de baja como propiedad sombra, filtro global de consulta y clave foránea restrictiva
RN-09	Todos los importes de una venta están en la misma moneda	—	Objeto de valor <code>Money</code> , con la moneda como constante del dominio
RN-10	El nombre de usuario es único y se normaliza	—	Agregado <code>User</code> , índice único y comprobación de forma normalizada en el motor
RNF-01	Dos ventas del último ejemplar no dejan existencias negativas	Decisión 2	<code>ConflictRetryPolicy</code> y el testigo de versión del motor
RNF-02	Un reporte de un período cerrado no cambia si cambia el catálogo	Decisión 4	<code>SqlSalesReportQuery</code> , que agrupa por los valores congelados
RNF-03	Toda lista que puede crecer viene paginada y con tope	—	<code>PageRequest</code> y <code>PagedResult<T></code>
RNF-04	De cero a sistema usable con una orden, sin pasos manuales	Decisión 1	Migraciones en el arranque y comprobaciones de salud encadenadas
RNF-06	Cambiar el almacén de binarios es un adaptador y una línea	—	Puerto <code>IFileStorage</code> con clave opaca
RNF-08	Cada atributo tiene clasificación y manejo exigido	—	Adaptador de seguridad y configuración de persistencia

Nota. Un guion en la tercera columna significa que la regla no exigió una decisión estructural propia: se satisface dentro de un artefacto que ya existía por otra razón.

Riesgos abiertos y límites aceptados

Un sistema no es fiable porque no tenga riesgos, sino porque los que tiene están escritos, acotados y asignados. Este capítulo recoge lo que se decidió **no** hacer y lo que se aceptó a cambio de otra cosa, con la misma disciplina con la que los capítulos anteriores recogen lo que sí se hizo.

La Tabla 11 da a cada riesgo un dueño y una vía de salida. Las dos columnas son deliberadas: **un riesgo sin dueño no está gestionado, y un riesgo sin vía de salida no está acotado.** Ninguna fila describe trabajo pendiente de construcción; todas describen propiedades del sistema tal como es.

Tabla 11*Registro de riesgos de arquitectura*

Riesgo	Impacto si se materializa	Estado	Dueño	Vía de salida
Las migraciones se aplican durante el arranque del servicio	Con más de una réplica, dos instancias competirían por aplicar el mismo cambio de esquema	Aceptado	Arquitectura	Mover la aplicación de migraciones a un paso previo del despliegue. Escrita desde la Decisión 1; mientras tanto, una sola instancia
La ruta de binarios no atraviesa la autorización	Una dirección filtrada sirve esa imagen a cualquiera y de forma indefinida	Aceptado	Propietario	Firmar la dirección o servirla desde un extremo autorizado. Cambia el mecanismo de servicio, no el puerto
La respuesta a un contrato mal formado no explica la causa y nombra el tipo interno del marco web	Quien integra recibe un detalle de implementación en lugar de un motivo, y el detalle describe la tecnología usada	Abierto	Propietario	Personalizar la respuesta de validación en el adaptador entrante, sin tocar el filtro de traducción
El borrado de un binario no participa en la transacción del motor	Una interrupción entre ambos pasos deja un binario sin dueño en el volumen	Aceptado	Arquitectura	Orden obligatorio: anular la clave y confirmar antes de borrar. Un binario huérfano es inofensivo; una clave rota, no
La carga de productos previa a vender podría no filtrar las bajas	Se vendería un producto retirado del catálogo	Cerrado	Arquitectura	El filtro global lo impide por omisión: saltárselo exige escribirlo, y solo lo hace la carga de uno por identificador, que debe resolver líneas históricas
Existe un solo adaptador entrante	La sustituibilidad del puerto entrante es propiedad del diseño y no hecho observado	Declarado	Arquitectura	Un segundo adaptador —una línea de órdenes o un proceso programado— la convertiría en hecho. Hasta entonces se enuncia con esa reserva

Nota. El estado «Cerrado» se conserva en el registro a propósito: un riesgo que desaparece de la lista al resolverse deja de enseñar por qué la solución es como es.

Deuda aceptada y decisiones aplazadas

Queda una decisión que **no es de arquitectura sino de negocio**, y que la Decisión 4 hizo visible en lugar de esconder. Con los nombres congelados, un producto renombrado o recategorizado entre dos ventas del mismo rango produce dos etiquetas distintas. Agrupar por

producto y por etiqueta devuelve **dos filas del mismo producto**; agrupar solo por producto obliga a **elegir** cuál de las etiquetas gana. Ninguna de las dos opciones es neutral.

La implementación de hoy **agrupa por la etiqueta congelada**: dos etiquetas son dos filas, y nada elige un ganador. La alternativa —mostrarlo todo bajo la etiqueta que congeló la venta más reciente del rango— tiene un defecto que la descalifica para este sistema: al ensanchar el rango, la etiqueta antigua desaparece y sus unidades se recuentan bajo un nombre que esa venta nunca llevó, de modo que **una venta nueva cambiaría lo que un período cerrado ya dijo**. Es exactamente el fallo que la Decisión 4 existe para impedir, llegando por la puerta de la venta en lugar de por la del catálogo.

Lo que queda abierto, por tanto, no es si se puede colapsar sino **si el negocio quiere colapsar**, y esa pregunta la responde quien renombra, no quien programa. La elección está escrita en el código como la cláusula de agrupación que es, a la vista y no disimulada dentro de una consulta, para que cambiarla sea un cambio de una línea y no una arqueología.

Consideraciones finales

El recorrido anterior permite una conclusión que no es evidente desde el diagrama: en este sistema, casi todas las propiedades arquitectónicas que se suelen enunciar como intenciones están sostenidas por un mecanismo que falla de forma ruidosa cuando se incumple. La regla de dependencia la impone el compilador a través de las referencias de proyecto. La prohibición de secretos por omisión la impone un arranque que se niega a continuar. El orden de arranque lo imponen comprobaciones de salud y no una lista de precedencias. El olvido del filtro de bajas lo impide un filtro global. Las invariantes de negocio las sostiene el motor además del dominio. Y la traducción de errores está en un punto único porque duplicarla exigiría escribir código que revisaría cualquiera.

Lo que no está sostenido por un mecanismo queda declarado como riesgo con dueño y vía de salida, y esa es la segunda mitad del mismo criterio. La tercera es el precio: cada atributo de calidad de este documento lleva escrito lo que cuesta —una venta que puede fallar bajo conflicto, un producto que aparece en dos filas, diez proyectos para un problema pequeño, una imagen que se sirve sin autorización y un sistema que hoy no escala en horizontal—. Un documento de arquitectura que omitiera esa columna describiría un sistema mejor del que existe, y sería la primera pieza de este proyecto en dejar de ser cierta.

La propiedad de diseño que permanece sin verificación empírica es la sustituibilidad del adaptador entrante: mientras exista un solo adaptador de ese tipo, la afirmación de que un segundo entraría por el mismo puerto es una propiedad del diseño y no un hecho observado. Conviene enunciarla con esa reserva, y no como si ya estuviera demostrada.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.). <https://doi.org/10.1037/0000165-000>
- Cockburn, A. (2005). *Hexagonal architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fielding, R., & Reschke, J. (Eds.). (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and content* (RFC 7231). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC7231>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Moriarty, K., Kaliski, B., & Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC8018>
- Nygard, M. T. (2011, 15 de noviembre). *Documenting architecture decisions*.
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>